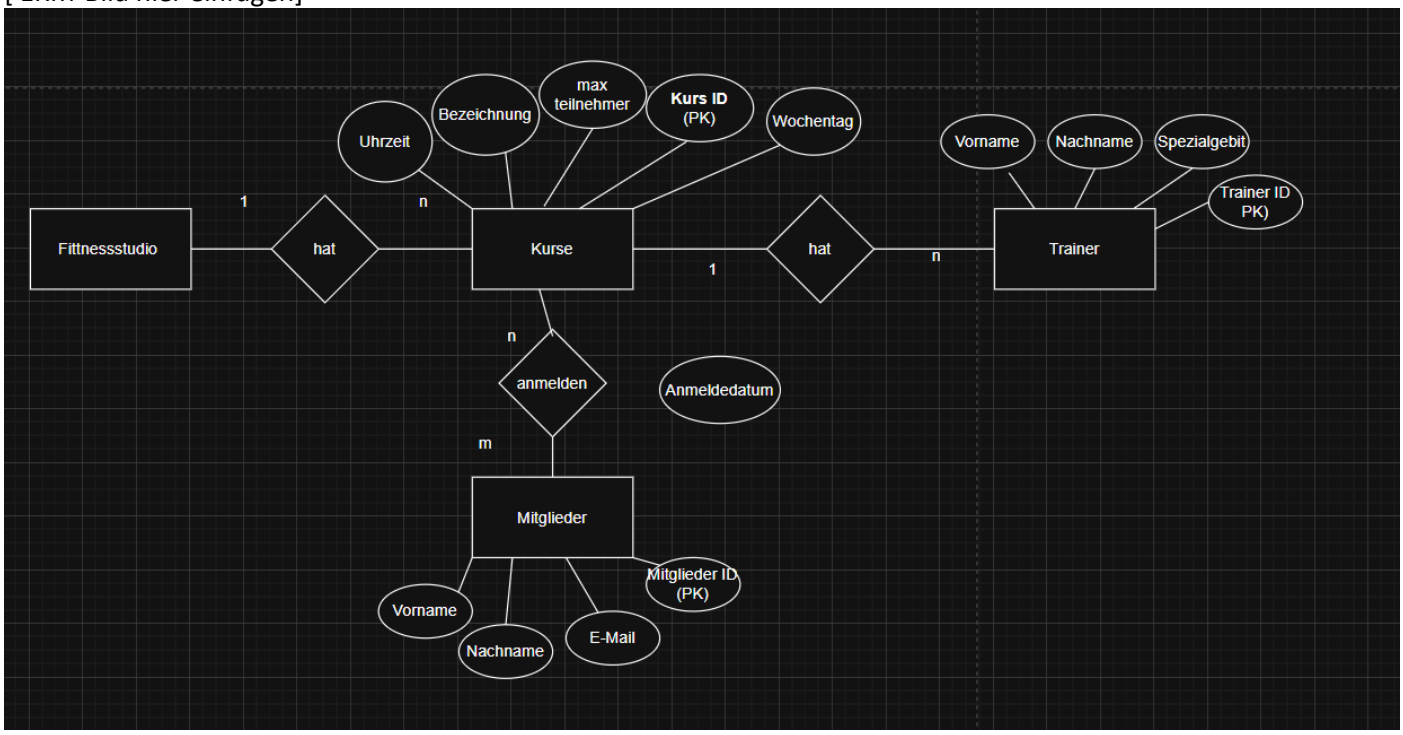
Name: Jakob Seeberger **Datum:** 08.04.2026 **Klasse:** 3AHIF

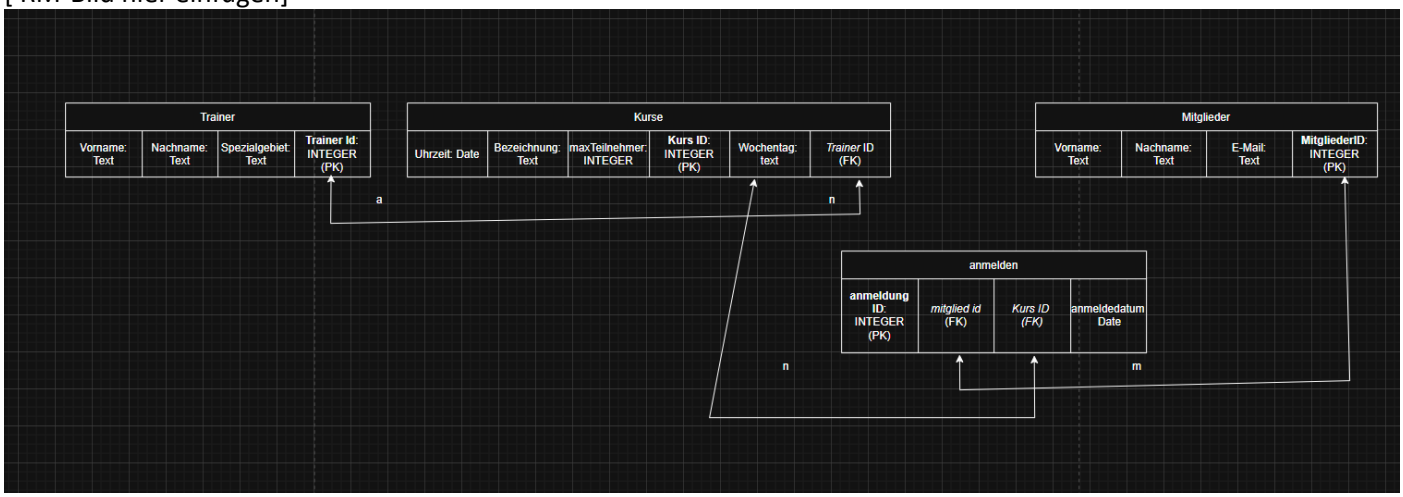
3. Grafische Leistungsfeststellung – DBI1
Thema: Fitnessstudio-Verwaltung (ERM, RM, Python/SQLite3, Anvil)

Punkte	**/ 100**				
Note	100-87.5 ☐ Sehr gut	< 87.5-75 ☐ Gut	< 75-62.5 ☐ Befriedigend	< 62.5-50 ☐ Genügend	< 50 ☐ Nicht Genügend
Unterschrift					
GitHub-Link					
Anvil Publish-Link					

Form	**Pun**
1. Form der Arbeit (Leserlich, vollständige Abgabe, sinnvolle Dateinamen, Ausführbar, Fehlerbehandlung)	
Szenario: Fitnessstudio-Verwaltung	
Ein lokales Fitnessstudio möchte seine Mitgliederverwaltung und Kursbuchungen digitalisieren. Es gelten folgende Rahmenbedingungen: • Das Fitnessstudio bietet verschiedene Kurse an (z. B. Yoga, Pilates, Spinning, Zumba). Von jedem Kurs werden Bezeichnung, Wochentag, Uhrzeit und maximale Teilnehmeranzahl gespeichert. • Jeder Kurs wird von genau einem Trainer geleitet. Ein Trainer kann mehrere Kurse leiten. Von jedem Trainer werden Vorname, Nachname und Spezialgebiet erfasst. • Mitglieder können sich für beliebig viele Kurse anmelden. Für jeden Kurs können sich mehrere Mitglieder einschreiben. Bei jeder Anmeldung wird das Anmeldedatum gespeichert. Von jedem Mitglied werden Vorname, Nachname, E-Mail-Adresse und Beitrittsdatum erfasst.	
1.1. Aufgabe 1 – Entity-Relationship-Modell (ERM)	**Pun**
Erstelle auf Basis des Szenarios ein vollständiges ERM: Füge das fertige ERM als Bild in dieses Dokument ein. **(draw.io-Export)**	___
[ERM-Bild hier einfügen] 	
1.2. Aufgabe 2 – Relationales Modell (RM)	**Pun**
Leite aus deinem ERM ein vollständiges Relationales Modell ab: Füge das fertige RM als Bild hier ein. **(draw.io-Export)**	___

Name: Jakob Seeberger **Datum:** 08.04.2026 **Klasse:** 3AHIF

[RM-Bild hier einfügen]



1.3. Aufgabe 3 – Python-Skript mit SQLite3 ★ KI-Tools erlaubt

Pun

Schreibe ein Python-Skript Nachname_Vorname_fitnessstudio.py, das:

1. Eine SQLite-Datenbank Nachname_Vorname_fitnessstudio.db erstellt
2. Alle Tabellen aus dem RM anlegt
3. Sinnvolle Beispieldaten einfügt:
   ≥ 3 Trainer • ≥ 5 Kurse • ≥ 6 Mitglieder • ≥ 8 Anmeldungen

1.4. Aufgabe 4 – Anvil-Implementierung

Pun

Implementiere die Fitnessstudio-Verwaltung als Anvil-Webanwendung (anvil.works).

Pflichtfunktionen:

1. Kursübersicht – Alle Kurse inkl. Trainername

Kurs	Wochentag	Uhrzeit	Trainer	Teilnehmer	Anmelden
Morning Yoga	Montag	08:00	Berger Anna	2/15	Anmelden
Power Lifting	Dienstag	17:00	Huber Markus	2/10	Anmelden
Pilates Basics	Mittwoch	10:00	Mayer Sophie	2/12	Anmelden
HIIT Cardio	Donnerstag	18:30	Schneider Leon	1/20	Anmelden
Abend Yoga	Freitag	19:00	Berger Anna	2/15	Anmelden
Rückenfit	Samstag	09:00	Huber Markus	1/10	Anmelden

2. Neue Anmeldung

Mitglied	Anmelden
Bauer Thomas	Anmelden
Fuchs Stefan	Anmelden
Schwarz Maria	Anmelden
Weiß David	Anmelden
Braun Nina	Anmelden

Bonusfunktionen (optional):

- Mitglieder in einem Kurs anzeigen

1.5. Aufgabe 5 - Abgabe

1. Nachname Vorname in der Kopfzeile eintragen
2. GitHub-Link oben eintragen
3. Anvil Publish-Link oben eintragen
4. Folgende Dateien ins Git-Repo pushen
   a. Nachname_Vorname_ERM.draw.io
   b. Nachname_Vorname_RM.draw.io
   c. Nachname_Vorname_fitnessstudio.py
5. Code hier einfügen

Name: Jakob Seeberger **Datum:** 08.04.2026 **Klasse:** 3AHIF

Nachname_Vorname_fitnessstudio.py

<hier code einfügen>

Anvil Frontend Code

<hier code einfügen>

```
from ._anvil_designer import Form1Template

from anvil import *

import anvil.server

from .Anmelden import Anmelden



class Form1(Form1Template):

  def __init__(self, **properties):

    self.init_components(**properties)

    self.lade_kurse()


  def lade_kurse(self):

    self.repeating_panel_1.items = anvil.server.call("get_kurse")


  def oeffne_anmeldung(self, kurs_id):

    alert(

      content=Anmelden(kurs_id=kurs_id),

      title="Mitglied anmelden",

      large=True,

      buttons=[]

    )

    self.lade_kurse()
```

Anvil Backend Code

<hier code einfügen>

```
import anvil.server

from anvil.files import data_files

import sqlite3

import datetime
```

Name: Jakob Seeberger **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
DB_NAME = "Seeberger_Jakob_fitnessstudio.db"



def get_db():
  conn = sqlite3.connect(data_files[DB_NAME])
  conn.row_factory = sqlite3.Row
  return conn



@anvil.server.callable
def get_kurse():
  conn = get_db()
  cur = conn.cursor()

  cur.execute("""
    SELECT
      k.kurs_id,
      k.bezeichnung,
      k.wochentag,
      k.uhrzeit,
      t.vorname || ' ' || t.nachname AS trainer,
      COUNT(a.anmeldung_id) AS teilnehmer
    FROM kurse k
    JOIN trainer t ON k.trainer_id = t.trainer_id
    LEFT JOIN anmeldungen a ON k.kurs_id = a.kurs_id
    GROUP BY k.kurs_id, k.bezeichnung, k.wochentag, k.uhrzeit, trainer
    ORDER BY k.wochentag, k.uhrzeit
  """)

  daten = []
  for row in cur.fetchall():
    daten.append({
      "Kurse": row["bezeichnung"],
      "Wochentag": row["wochentag"],
      "Uhrzeit": row["uhrzeit"],
```

```
    "Trainer": row["trainer"],
    "Teilnehmer": str(row["teilnehmer"]),
    "kurs_id": row["kurs_id"]
  })

  conn.close()
  return daten



@anvil.server.callable
def get_freie_mitglieder(kurs_id):
  conn = get_db()
  cur = conn.cursor()

  cur.execute("""
    SELECT m.mitglied_id, m.vorname, m.nachname
    FROM mitglieder m
    WHERE m.mitglied_id NOT IN (
      SELECT a.mitglied_id
      FROM anmeldungen a
      WHERE a.kurs_id = ?
    )
    ORDER BY m.nachname, m.vorname
  """, (kurs_id,))

  daten = []
  for row in cur.fetchall():
    daten.append({
      "Mitglied": f"{row['vorname']} {row['nachname']}",
      "mitglied_id": row["mitglied_id"]
    })

  conn.close()
  return daten
```

Name: Jakob Seeberger **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
@anvil.server.callable
def anmelden(mitglied_id, kurs_id):
  with data_files.editing(DB_NAME) as db_path:
    conn = sqlite3.connect(db_path)
    cur = conn.cursor()

    cur.execute("""
      INSERT INTO anmeldungen (mitglied_id, kurs_id, anmeldedatum)
      VALUES (?, ?, ?)
    """, (mitglied_id, kurs_id, datetime.date.today().isoformat()))

    conn.commit()
    conn.close()
```